

DESIGN PDF EDITION

The Design

What it is, how and why it was created

Core idea: a computable semantic corpus with evidence-bound knowledge layers - not just a vector-search wrapper.

Contents

Opening facts before the design process	3
The Design. What it is, how and why it was created.....	4
1. The Optimizer	5
2. The Interpreter	8
3. The Validator	9
4. The Normalizer	11
5. The Corpus Builder	14
6. The Edit Suite	17
7. The Orchestrator.....	18
8. The MCP Server	19
9. The Semantic Control Kernel	20
1. The "Agent does everything" MCP roundhouse.....	20
2. The "Agent gets a handbook with buttons" approach.....	21
3. The "bottom up tool chain" Agent entry portal.....	21
4. The semantic control kernel.....	22
9. The Client Frontend	26
The Query Agent.....	26
The Taxonomy Agent.....	28
The Ontology Agent.....	32
10. Use Case Examples	39
Media Worldview Analysis	39
Peer Review As Computable Knowledge	40
Legal Discovery And Case Memory	41
Corporate Audit, Procurement And Fraud Detection	41
Intelligence And OSINT Dossiers	42
Investigative Journalism Over Large Archives	43
Literature, Storycraft And Editorial Surgery	44
Theology, Scripture And Tradition Mapping.....	45
Medical Record Review And Care History.....	45
Personal Life Archive And Memory Machine.....	46
University And Research Stack	46
Education And Curriculum Mining	47
Product Support And Field Failure Memory.....	48
Regulatory And Compliance Memory	48
Software Architecture And Codebase Archaeology	49
Market And Competitor Intelligence.....	49
Cultural Pattern Mining	50
Memory Core For Evolving Agents	50
Why These Use Cases Fit The Machine	52

Opening facts before the design process

Before we begin the actual design process description, here are the hard facts that might interest most people:

For Version 1.0, there is one pre-installed Demo Database and two official Sample Database packages belonging to the Ontology Machine. The Default Demo is part of the install payload. The larger Enron and purchase/invoice/delivery samples are official release/sample assets and don't need to be shoved into every installation folder if this would make the installer stupidly large.

1. The pre-installed Default Demo Database
2. The Enron 1k sample Database
3. The 800/800/800 purchase/invoice/delivery triplet Database

Total pages within all Databases: 4652 Total processing time: about 3 days

Embedding tokens: 9 million - 16 cents - text_embedding_3_small

Process tokens:

GPT 5.4 mini: 12 million input tokens (9 Dollar), 7.5 million output tokens (34 Dollar) GPT 5.4: 17.5 million input tokens (44 Dollar), 12.3 million output tokens (185 Dollar)

Total cost: 272 Dollar

Average price per page ingested: 5.8 cents

(Model Prices 06.2026)

The two large Databases are not intended for use within the Ontology Agent. They are just query demos to show how the Query Agent works over larger corpora. Also, those two DBs have some flaws that are deliberately left this way for the user to see what those flaws mean and how they affect retrieval and quality. And yes, also because I am too lazy to make another almost 3 day run with the updated taxonomy for something very few people actually will have look at.

The fully verified, with a clean and deep ontology layer is the pre-installed demo DB.

The Design. What it is, how and why it was created

Document route map

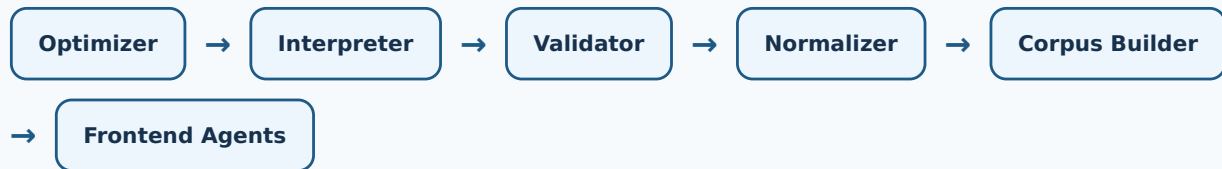


Figure 1. Main processing route described by the design document.

The reason why the Ontology Machine is designed the way it is, is not a coincidence.

The design process and its functionality were a process of iteration, refinement and dead ends.

To understand the rationale behind its Version 1.0 form, the reader needs to understand its design process.

The following descriptions can be understood in the way the data takes while is being processed. Which means the first part, the Optimizer, is where the data enters the system the first time. Every subsequent chapter represents the next step the data takes downstream until it reaches is final destination, the Client Frontend.

1. The Optimizer

The basic function of the Optimizer module is to take in heterogeneous data in the supported digital text and image-like formats and distill it down to a single basic JSON form. Besides the fact that this needs various plugins to make the file readable for The Machine, it is not immediately obvious what JSON format the basic extract should have.

In previous versions of this module, there was already a bit of pre-semantization going on with light heuristics and hardcoded filters to make the output more business-data friendly. However, as it turned out, every bit of pre-semantization in this stage made the downstream LLM work more and more costly, because it became clear that it is way more compute intensive to re-interpret wrong semantics than to work on a non-semantic blank slate. And also, because business data is just a small subset of data than can be ingested and not even the most interesting one.

In previous versions, the OCR part was tested with Tesseract, Paddle CPU, Paddle GPU and finally with the newest Paddle 0.9b vision language model. Over test runs it became clear that fast OCR methods like Tesseract and Paddle CPU paid for their speed not only with a loss of accuracy, but also with weaker general extraction capability when the input had a non-standard form factor. The loss of accuracy was so severe that it became clear that such methods were unsuitable for the purpose. The somewhat useful tool Paddle in its GPU version turned out to be usable but too edge-case sensitive. Only the Paddle VLM turned out to be flexible enough to also handle edge cases well enough to produce usable output for downstream consumers.

But this brought a whole new problem of its own. The Paddle VLM used up to 9GB of VRAM per worker and had a silent sensitivity to VRAM overflow in which the extraction became corrupted without any external notice. The reason why development focused on local extraction was simple token economics. Every step that could be done locally would save tokens and ultimately money. But the VLM OCR also meant that this tool, and ultimately the whole Ontology Machine, would remain a toy for those already in possession of a large RTX-like GPU.

This was the ultimate reason why, during the development of The Machine, the product route tossed out local OCR as the default answer and favored one simple LLM call instead. Digital extractions are still local and deterministic, but image-like extractions are delegated to a small online LLM call. There may still be old local OCR code history or experiments around, but the V1 design decision is clear: no GPU OCR requirement for normal use.

The end result of what the Version 1.0 Optimizer does is to deliver a simple form:

```
"ocr_reference": {
  "blocks": [
    {
      "id": "",
      "type": "",
      "value": "",
      "position": {
        "page": ,
        "paragraph_index":
```

free of domain semantics and with locatable position identifiers in a minimal, token-efficient format. No need for fancy pre-semanticization shenanigans and, as it turned out, way smoother to digest for the downstream process. If the user likes to keep their OpenAI OAuth key for this stage, GPT 5.4 mini is strong enough for the task and does not eat away the credit usage as fast as a frontier model does.

In general, the Optimizer in its version 1.0 is a slim, non-heuristic module that has only one job: deliver the blank JSON for the supported input routes, while staying away from early semantic judgement.

But here's the real question: Why does the optimizer not support Excel and CSV as a real V1 ingestion route? Well, in its first iterations it had an extra table route dedicated to extracting Excel-like structures. However, Excel and CSV aren't just another form of text file. They are basically mathematical structures condensed into human-readable graphical form. Treating them like normal text files would mean dumping the file raw into JSON as it is, creating huge JSON that exceeds any reasonable token efficiency even with single-sheet files in no time, let alone multi-sheet files beyond 100 rows. The more complex a sheet, the less likely a raw file would be usable downstream. Deterministic pruning only gets you so far in compacting a raw extract.

At some point, one reaches the insight that deterministic extraction is not a viable way to handle Excel and CSV if one wants true universality and edge-case coverage. That means there will eventually be heuristics needed to compress the files and pre-structure them in a way that lets the interpreter do its job in a reasonable amount of time and with a reasonable amount of token usage.

But as the reader can see, there is no usable Excel path within the optimizer module, which means I failed to come up with a viable solution that works within the limits of the other raw extractors when it comes to token usage while also preserving the ground truth as a readable page image. The system may know these suffixes at intake level and can put them into unsupported/error handling, but this is not the same as having a working ingestion route. I even tried to visually compress big sheets into small alias structures, like a visually encoded QR-code form, so the LLM does not have to churn through raw JSON and can instead use its visual systems downstream, but no matter what I tried, a working and stable Excel route remained elusive. So the only way to proceed with the whole system was to cut out Excel capabilities altogether for V1. Maybe someone else can come up with a viable way to integrate table-like files into The Machine structure. The basic raw file format contract should be working here too.

It should be noted that I purposefully omitted the "Blueprint" way of table extraction, which means creating a pre-made mask used to extract the raw JSON from a single file design. While this is certainly a way to go if the file design is singular and has hundreds or more to batch extract, it does not solve the underlying problem of being able to extract reliably from any form, and building a blueprint creator route just for the possibility of having a standard Excel format seemed to me like a waste of time for the benefit it seemed to provide. However, this may change if a company wants to use The Machine extraction on its own documents and wants to include Excel files. Depending on the similarity and amount of files, a custom blueprint route still is a viable solution.

2. The Interpreter

The basic description of what the interpreter does is to answer the question "what do you see?" and to deliver the answer in a way that minimizes the see/hallucination ratio natural to an LLM system. Hallucinations cannot be prevented, so they need to be managed and reduced to a level that does not hurt the basic process.

Again, this was an iterative process. The main difficulty was to find out what the best balance between output format, output stability, system prompt strictness and variability is. In order to figure this out, it's not merely about telling the model "do X and not Y" but rather creating a prompt form that allows the model to work as freely as possible while reducing ambiguity in every domain to a minimum. The balance can be seen in the prompt structure as well as the model output structure:

```
\The Ontology Machine\02 - Interpreter\llm_interpreter\prompts
```

Interestingly, as it turned out, forcing the model in this stage to output a strict JSON schema seemed to visibly limit its expressiveness. Therefore, the model is guided into a JSON-like structured object, but not treated like a little deterministic parser that can only speak one schema. The system validates and repairs the slight variability this causes in the model output in later stages in favor of a richer semantic output.

Also, there are two distinctive modes the model works in, depending on which format is ingested. If it's a scan or an image, the model is explicitly told to treat the raw JSON as secondary evidence and the image within the call as the prime source of truth. If the file is born digital, this reverses and the model is told to treat the raw JSON as the ground truth and the vision part of the call as supplementary. The reason why it was designed like this lies within the nature of the LLM itself.

Because the OCR is done with a weaker model, we must assume that it contains hallucinations at a higher rate, not only because the model is weaker, but because it works with a blank slate and therefore has to "create something out of nothing", which is an incentive for all models to hallucinate. We can't prevent that, so we work with it. The OCR raw extraction from a scan may or may not be faulty, and the LLM is told to reconcile a scan raw against its own vision call, or reconcile its own vision call against the born digital raw. Either way, the model does not have to invent anything new. By introducing two ways of extracting information from a source, raw JSON and the vision call within one interpreter request, we are not just inflating token usage, we are creating the conditions in which the model is least likely to hallucinate, since there are no blank slates anymore for the LLM to fill in with its imagination, just comparative actions based on the assumption that one source can be faulty. This does not prevent hallucinations, but reduces them noticeably and makes them manageable for the following validation step.

3. The Validator

The validator is the gateway that checks the model output against a set of rules that further reduces the chances that faulty information not derived from the ground truth passes down the pipeline. Again, it's not about preventing every mistake, but about reducing the likelihood of silent faults propagating into the system.

As with the Interpreter, the validator also has two validation modes depending on where the data originated. If it originated within a scan or image source, we naturally lack objective ground truth markers to validate against and so we need to use a little trick.

Within the interpreter call of the scan route, the model is pushed to replicate the important visible claims and values from the structured JSON into the `free_text` section too. That means the document basically exists twice in one model answer, but not as a perfect byte-for-byte clone. Again, this is not because we want to inflate token economics, but to force the model into another comparative mode in which it needs to make sure that both sources contain the same information. Within the validator for the scan route, the validation step then checks presence, important values, row anchors and numeric/date claim survival against `free_text` and raw evidence. So the claim is not "the whole document is duplicated perfectly", but "the relevant structured claims did not float away from the evidence layer".

This is not a guarantee against faulty information, but a pretty high barrier against it still silently slipping through.

If the information originated as a born digital file, the validation step incorporates strict numeric claim verification, which means that all numeric claims like currencies, dates, amounts, etc. that were present in the raw JSON must also exist within the interpreter output.

There, we must make use of light heuristics to normalize the raw values against the interpreter output, since row merges, mojibake and whitespace can make the raw value differ from the LLM-interpreted output, and so we need to tune the validator heuristics to a certain degree to catch as many variations as possible. Again, this is not about perfect security, but about reducing faulty information. Working with non-deterministic output has its own challenges.

For edge cases and ambiguities, the validator can set a "review flag" with a review reason that is written into the DB as a warning that says where possible ambiguities and weak information may have occurred. This is not only hardcoded, but also uses the interpreter confidence output and model response to determine if a review flag needs to be set.

Typical review flags occur if the taxonomy has too many "other" classifications that hint towards non-optimal coverage, if the image quality is low so the model lowers its confidence score, or if there are mixes of symbols, tables, rows and certain other combinations that represent edge cases that haven't produced a hard fail within the validator but have "soft edges" that need later post-db creation treatment if the user wants

a "review-free Database". Those signals aren't lost and are marked within the Database itself.

The validator is not foolproof and lacks distinctive capabilities like hard validation against phrases, words or claims from born digital documents, since the LLM tends to rephrase text, which makes it impossible to use the raw JSON for validation as strictly as I do with numeric claims. So I decided to keep numeric claims as a hard validation barrier and not wording. It's a tradeoff I made since it would certainly be possible to also use the `free_text` trick within born digital documents, but that would increase token usage and would not provide the same benefit as it does in the scan route. In the born digital route, I am certain that the raw extract is 1-to-1 source truth, so the LLM has no ambiguity to deal with other than its own.

This is not a side effect but part of the design. Error Cases and review flags are not trash output that should be hidden because it looks ugly. They are visible uncertainty artifacts. If a file fails, the Error Case should preserve enough of the request, artifacts and reason so the user or developer can understand what went wrong. If a file passes but smells weak, the review flag keeps that weak smell computable inside the Database. The point is not to pretend that the Machine never fails. The point is to make failure and uncertainty queryable.

Generally speaking, the validator should not be seen as a hard truth gate but as a reducer of ambiguity, a marker of what may be wrong and a gate that can show when a certain type of document is not suited for ingestion into the current pipeline system.

4. The Normalizer

Within the normalizer there are two different domains. The first is the LLM call where the model is told to take the structured interpreter output, hold it against the current taxonomic projection normalization mask and transform the interpreter keys into the standardized language of the taxonomy.

It's important that not the whole structured output is treated that way. The normalizer uses top-level promotion slots that are provided through the taxonomy to create a "normalized first" SQL view that can be searched fast and efficiently while still covering the essence of the underlying richer structured interpreter output. This saves both token usage and search time. The normalized first view is there so the Agent can gather fast insight into the whole content without needing to use `get_document` for everything. Together with adequately sized embedding chunks, this provides the fast entrypoint into the Database that does not suffer from AI variability, which is the prime reason the taxonomy exists in the first place.

Pre-Taxonomy Pipeline Designs showed the clear weakness of using an SQL-like system that is filled with fluid LLM semantic classifications. When there was no canonical top-level structure, the query agent struggled heavily to get clear and, most importantly, complete insight into what is actually written. The embedding helped to remedy this in a certain way, but cosine similarity is no real substitute for a coherent naming convention that holds true across the database. So, introducing the taxonomic top-level structure was a "must have" in order to make the whole Machine work as intended.

The second part of the Normalizer is the whole Semantic Release Management System itself. But here we face the problem of language. Taxonomies are language-bound by nature and a Spanish, Chinese or English taxonomy aren't compatible, so the whole system, if built on a fixed language, would need a whole taxonomic system for each language separately, which would not be feasible or practical.

Instead I intentionally split the system into two layers and made use of the LLM capability to work language-agnostically. The "machine language" and control layer is written in English in a way that lets the LLM map any language onto the control layer and structure the output in any given language. This is simply done by two prompt tweaks:

1. During ingestion, the model is told to map concepts rather than language, which can be expressed in any language.
2. During the query/creation/mining process within the Client Frontend, the model is told to reply in the language the user is speaking to the model.

Important here: this does not mean that the normalized Database canon itself is written in any output language. In V1, the internal control language of the normalized layer is English. Source documents may be German, Spanish or whatever, but human-readable normalized summaries, tags, notes and free_text are translated into the English control layer while stable identifiers, dates, amounts, names etc. stay stable. The user, however, can query and discuss the Database in their own language through the Frontend Agents.

This can be seen in the pre-installed SampleDB. The book ingested is in German, but can be queried, mined and discussed in any language the user might speak. In essence, the user would never need to read the control layer and does not need to speak English to make use of the full Machine capabilities if they simply work through the chat within the Client Frontend (apart from the config to provide inference provider credentials and model choices).

The Semantic Release Management and its product, the "semantic release" and its projections, are a hierarchical model of concepts. It can be understood as a tree that defines how concepts are organized into smaller segments of sub-meaning chunks. An invoice may incorporate different meanings like "transportation", "health" or "legal" and may have "contractors" attached to it, delivery notes or phone numbers, tracking codes or names. All those things form a hierarchy that is collected under a main "meaning" topic. The taxonomy can hold as many meaning chunks as one wants and needs to cover a certain terrain that the ingested documents represent.

In the early design phase this taxonomy was a hardcoded list that was designed to cover the basic terrain of everyday use. The remnant can be seen in the Master-Default-Taxonomy:

```
\The Ontology Machine\05 - Corpus Builder\config\semantic_release.default.json
```

This list with its 11 default projections is the default preset one can use to classify ordinary documents like invoices, home letters, diary entries etc. But due to its broad nature, it's not suitable for specific document types that should be captured in full detail. It's a demo Taxonomy that shows how the main structure came to be.

In the later stages of the development it became apparent that custom taxonomies cannot be created manually through an editor even with a guided path. The landscape of meaning is just too vast to capture by a single human mind, let alone to structure it into the strict release format needed for the normalizer to work with. And so, the "Taxonomy Agent" within the Client Frontend was born and is described in its own section.

It's important to understand that the real challenge is not to create the taxonomy, but to guide the model to an understanding of what its purpose is and what it should represent. The longest task therefore wasn't to create the code, but to figure out how to get the desired richness from the interpreter module into the normalizer model call and out as a usable normalized canonical shape for downstream progression.

The Normalizer prompt is assembled dynamically from the active Semantic Release projection. For each normalization call, it injects:

- the active taxonomy profile ID
- allowed document_type, category, and subcategory codes
- allowed field_codes, row_types, and cell_codes
- each allowed code's description
- the projection's promotion rules as a materialization contract
- dynamic output schema examples matching the active projection
- the raw Interpreter classification hint
- the current structured.json payload

The static part only defines general normalization behavior and output discipline. Domain-specific behavior is not hardcoded globally; it comes from the active taxonomy/projection through codes, descriptions, aliases, and promotion rules.

Due to token economics, a specialized "normalization_guidance" that is dynamically rendered into the normalizer prompt for every projection used was omitted. In theory, such guidance, if created by the custom taxonomy creation, would increase the semantic precision of the normalized layer, but would also noticeably increase the token cost per page ingested by about 30% if used in its optimal shape. This might not sound like much if we consider that the normalizer uses a weaker and cheaper model, but there needs to be a line drawn, otherwise feature creep sets in at every corner. If token prices come down, there are many more such improvements possible across the whole Machine, but until then, we refrain from pursuing perfection in this field.

The Normalizer holds the authority over what can be represented in the canonical state, which releases are compatible, which databases can be merged, what fingerprint is validated and which semantic release can be activated. It is designed to keep any database corruption at bay by enforcing a list of checks and rules for how a semantic release is allowed to work, mainly to retain the complete and unbroken evidence path from raw ingestion to Database creation. All downstream actions validate against the Normalizer output in order to guarantee traceability, visibility and debuggability.

5. The Corpus Builder

Here all created meaning-bearing artifacts come together to be materialized into the final database. The core structure is page-wise and structured as a hierarchy, where every mutating step builds on the previous one to form a traceable stack of what becomes the corpus, the self-contained database that holds the materialized state and can be used on its own. The artifact tree in which the corpus is created is not just decoration or dump material. It is the audit, request, error case and rebuild surface around the Database. The corpus can be used without external dependencies, but the artifact tree is the place where a human or developer can see how the corpus came into existence. The SQL corpus is structured in the following way:

At the ground level we have the `page_image`. This is the PNG-rendered version of the original input format. It is rendered as an image so it is immutable, even if someone tries to change everything that follows, so everything can be traced back and compared to the original input. In V1 the page image has two roles that must not be confused. The Artifact Tree `page_images` folder is the rebuild and audit surface. The DB table `document_page_images` is the direct evidence back-link surface inside the corpus itself. It is not merely a fast UI cache. It means that a DB answer can point back to the visible page evidence without needing to trust the JSON layers alone.

The next stage is the raw extract. This is the result of the Optimizer module and the basic JSON extract on which every following step is grounded. It remains in the Database as the first machine-readable ground layer that can be directly queried by the Agent. It has no domain semantics, just raw extract.

Then comes the structured data on top. The structured output of the interpreter module is the first semantic base layer that incorporates the rich but fluid LLM-created semantic interpretation of the raw data. That includes:

```
"level": "section",
  "title": "",
  "page_reference": "",
  "parent":

"segments": [
  {
    "segment_id": "",
    "unit_kind": "",
    "page": ,
    "sequence": ,
    "section": "",
    "label": ,
    "text": "",
    "function": "",
    "confidence":
```

This is used to describe the content and context of the page in question in a semantic manner so its core meaning is preserved and condensed. The goal is to preserve the meaning-bearing content, but this does not make the structured layer completely raw-

copy-proof. The raw extract and the page image remain the evidence truth. The structured layer and its values are free-formed by the LLM and do not possess a rigid taxonomical format, so the expression and extraction can be as complete as possible, while still belonging to the interpretation layer and not to the immutable ground truth layer.

The next step is the canonical normalized layer. Here the structured data is transformed into the canonical database language provided by the taxonomy and its projections, which includes "content", "fields", "rows" and a free_text section for the summary. All keys, types, categories and subcategories are canonical for all documents ingested under the active semantic release, which means one SQL query captures all pages in the database in the exact same way. So semantic fluidity does not create noise that may mask corresponding information asked for by the query.

The normalized layer is intentionally condensed and does not carry the full payload the structured data holds. This is due to two reasons:

1. To reduce noise within the top level view and only provide the core information present within the document.
2. To reduce ambiguity for the search agent and speed up the query process.

There was a tradeoff during the design process in which the Query Agent was tested against a range of different ways to query the Database. It turned out that augmenting the LLM request with validation rules to force more precise answers over a broader normalized range would visibly degrade the Agent's capability to freely move through the corpus.

The fewer barriers and the less ambiguity there were between the Agent and the Corpus, the better the Agent seemed to work within the whole structure. So it was decided to reduce the normalized view to a condensed but still precise format for the Agent to start with. From here, and due to the fast nature of the initial query, the Agent can form a strategy on its own for how to proceed with the query, is not hindered by too much noise or too many barriers, and can move down to the structured data only if the initial request resulted in precise matches. In this sense, this creates a fast search tree optimized for meaning extraction.

This is also why the DB should not be understood as just a search index. The Corpus DB is an evidence carrier. It holds documents and payloads, page images, evidence atoms, promotions, extracted fields and rows, source document groups, classifications, embeddings and later ontology lenses in one materialized object. This means an Agent can search, but also walk back from a claim to the DB row, from the row to an evidence atom, from the evidence atom to the page image and from there to the visible ground truth. The DB is not the original truth, but it carries the evidence links that make the materialized truth inspectable.

The embedding vectors are then written. There are two ways embedding vectors can be searched. One is the document-level embedding in the embeddings table, built from the compact retrieval text. The other is the chunk layer in embedding_chunks, where the document is split into adequately sized searchable pieces. Search can prefer chunks when they exist and fall back to document-level vectors when they do not.

At this point, the basic database is written. But the corpus builder includes other layers that remain empty and will be used by a post-creation mining step.

The first is the Base Graph. Since the ingestion is page-wise, the pages arrive as individual materialized documents. The corpus already carries `source_document_id`, `source_uri`, `page_index` and similar source identity markers, so the Base Graph does not need to guess this from filenames. Here, the ontology agent can run the deterministic kernel step that creates the Base Graph from those existing markers, connecting pages together, filling `source_documents` and `source_document_pages`, creating `base_unit` and `page_unit` structures and adding the first relations that are directly extractable without an extra LLM call.

This completes the Database as a closed system ready for use.

But the corpus builder also adds the empty ontology layer that can be used later by the ontology agent to build new interpretations on top of the base structure. This will be presented in the section belonging to the Ontology Agent further down.

6. The Edit Suite

The Edit Suite is a tribute to visibility and Nerdism. While not strictly necessary in its current form, it provides the user with many system config values that can be changed through visible owner surfaces. It is important to note that while a wide range of values can be edited here, it is generally not advised to play around blindly. This is not just random raw config hacking; many surfaces are owner-contract backed, validated and tested. But the user should still treat it as an advanced working surface, not as a cute settings menu where every knob is harmless.

The Edit Suite can be accessed through the Orchestrator main window with the button "Open Edit Suite".

The main reason why the edit suite exists is that the normalizer section provides a way to manually tune the current active semantic release and other owner-level config surfaces. It is highly advised to read and thoroughly understand how edits must be performed to result in a usable semantic release. For this, there is the `Semantic_Release_Edit_Contract_Handbook` provided with the documentation, which explains how the semantic release is structured and how it should be safely edited. This is not a trivial task, and any attempt to manually refine a release should only be done after a backup of the original is created. The Edit Suite should make sure that no corrupting edit results in a defective Database, but better safe than sorry because this route is powerful and therefore not a toy surface.

And most importantly: The Edit Suite is not fully field tested and debugged. There might be dials, knobs or workflows in there that produce unwanted side effects resulting from bugs not discovered yet and workflows not fully tested end-to-end.

7. The Orchestrator

Here you can find the main control plane for the document pipeline and its debug routes. It is not the place where literally every process of the whole Ontology Machine lives, because the Kernel, Taxonomy Agent, Ontology Agent and Client Frontend grew into their own control surfaces. Within the Orchestrator you can run the pipeline, manually run modules in debug mode and inspect the output, write the API credentials and model selection for the pipeline modules, create an empty artifact tree, reset logs and error bundles, etc.

The Orchestrator is for people that want control over the document-mainline process and want to have more visibility of what's going on under the hood of The Machine. It owns the ingestion operation, artifact tree handling, module debug host and many pipeline-side settings. It is the "head of operations" for this mainline, but not the boss of every product capability. Taxonomy creation, long-running Kernel workflows, Ontology work and conversational database usage are their own governed surfaces around the Orchestrator, not simple Orchestrator submenus.

During the design process, this module was the go-to for debugging workflows outside the control Kernel, which came into existence way later in the process. It is also a remnant of a time where it seemed appropriate to pool together the submodule controls so it became less complicated to manage the actual workflow. It must be noted that, at the beginning, the idea was to create just single self-contained modules that ingest the output of the previous one without any inter-module orchestration. This was due to the fact that AI could not reliably code over a larger codebase with pre-GPT 5 models. So breaking them down into smaller pieces seemed appropriate at the time. The modern reading is a bit sharper: the Orchestrator owns the document-mainline operation and debug visibility, while Kernel and Frontend own their own workflow/dialog/user surfaces.

Since GPT 5, things have changed. While the current module design turned out to be well suited for AI code editing, it also became way easier to reason across all modules and create combined workflows like the Orchestrator and the control kernel. The isolated runtimes and wheelhouses for each module are partly historical, partly field-ready boundary. Today it might be possible to fold all modules into a single runtime, but since the system works like it does, such a refactor seems to be too much hassle for the benefit of saving some disk storage and would also weaken some of the module boundary clarity.

8. The MCP Server

The MCP Server is the local tool bridge of The Machine. It is the part that allows Agent surfaces to reach into the installed system in a structured way instead of being just a chat window that can talk about things but not do anything.

Technically, it is a local stdio control plane. It speaks MCP over standard input/output, exposes a tool catalog and returns structured tool responses. This means it does not need a big external network surface to be useful. It sits on the same machine as the rest of the system and gives the Agent a local, inspectable list of actions.

The important design idea is owner delegation. The MCP Server exposes tools, validates their arguments, checks permission levels, and then routes the actual work to the module that owns the truth for that action. Normalizer work goes through Normalizer contracts. Corpus Builder work goes through Corpus Builder contracts. Orchestrator runtime work goes through Orchestrator contracts. This keeps the system from turning into one huge shared state soup where every component writes into every other component because it knows a file path.

So the Agent sees a practical tool surface, but the module ownership remains intact underneath. This is the useful compromise. The Agent gets hands, but those hands are not random file access. They go through named tools, schemas, permission policy and owner contracts. That is what makes automation usable without turning the Machine into a pile of untraceable side effects.

This also explains why the MCP Server carries its own permission concept. Some tools are harmless read or inspection tools, others can change runtime settings, activate releases, start pipeline work or touch credentials. Those actions are not the same kind of power. The permission layer exists so an Agent surface can be useful at different levels without every possible tool being equally exposed all the time.

The Semantic Control Kernel is also surfaced through the MCP Server. The Kernel owns workflow semantics, progress, resume states, recovery and long-running decisions. The MCP layer exposes the Kernel-facing tools and calls the Kernel through its local subprocess contract. This is how the Taxonomy Agent can see workflow actions as tools while the actual workflow state and decisions stay inside the Kernel.

In the larger design, the MCP Server is the controlled hand of the Agent world. It does not need to be dramatic. It is a transport, catalog, permission gate and delegation layer. But without it, the Agents would either remain mostly conversational, or they would need direct knowledge of too many module internals. The MCP Server gives them a proper handle on the Machine while keeping the owner structure readable.

9. The Semantic Control Kernel

This is where most of the automation happens outside the pipeline ingestion process. Code-wise, this is basically a system within a system that maybe carries even more complexity than the whole rest of The Machine. This is also the last piece of the Ontology Machine that came into existence after many failed trials to create a manual Taxonomy creation route, then a semi-manual route, and then the insight that a controlled system is needed that takes away every possible ambiguity from the process of semantic release creation.

The problem the control kernel addresses is how to reliably execute all the needed creation steps for a usable semantic release. Under the hood of The Machine and without the control kernel, these steps are all present and workable but scattered across the modules and MCP surfaces. The real insight was not that "the Agent is stupid". The insight was that chat context is the wrong place to store workflow truth. A workflow needs state, progress, dialogs, resume points, confirmations, recovery and receipts. If all of that lives only in the Agent context window, it turns into noise and eventually the Agent no longer knows where it is. The control kernel was not designed in one fell swoop but, like all parts of The Machine, in small iterative steps. In this case, there were 3 versions before the final V1 design.

1. The "Agent does everything" MCP roundhouse

The MCP Server had, at one point, about 120 exposed tools for every process The Machine was capable of. All of them had their own tool descriptions. The idea was to give a "General Pipeline Agent" permission-layered access to all of them with a single prompt-attached tool list. Admittedly, this list was huge and counted roughly 80k tokens per tool list call.

The Agent was then prompted to achieve a certain outcome like kicking off a pipeline ingestion run or creating/modifying/activating/resetting/merging or analyzing the active semantic release. In theory, and if the Agent were perfectly capable of following instructions, this would have worked. But in reality, the Agent was barely capable of executing 3 MCP tool calls in a row without maneuvering itself into a dead end. This was not because the Agent had no intelligence, but because the task was cut wrong. The Agent had to be planner, state memory, tool router, error handler and user explainer at the same time. The remedy was an ever-growing instruction prompt dealing with more and more Agent edge cases which, at one point, created a token budget well above 200k for every instruction chat turn. Suffice to say that this was not an efficient way of controlling The Machine because despite the huge token budget, the Agent still wasn't able to reliably execute medium-difficulty tasks like merging a database. This approach was abandoned.

2. The "Agent gets a handbook with buttons" approach

This iteration was closer to the current design, but with a caveat. The agent no longer had a giant tool list to choose and work from, but a detailed description of every desired workflow the user may want to execute. These workflows, if the Agent recognized the corresponding user intent, then exposed only those MCP tools the agent needed to accomplish the workflow.

Again, in theory, reducing the Agent's tool exposure should make it easier for the Agent to choose the appropriate tool call chain. But again, in reality the old problems of the first approach just came back in the exact same way. However, the underlying problem became visible. It wasn't so much that the Agent was too unreliable, but that the tools themselves weren't designed to be executed in an obvious "this is the next thing to do" way. Due to the nature of chat rounds and context retention, the tool calls piled up within the agent context window to a point where the Agent lost a clear overview of what it was actually trying to achieve because, within the Agent context, tool calls accumulate as noise. But removing the tool call history from the Agent context and letting it live only within the current turn robbed the Agent of the knowledge of where it was in a given workflow. So the system needed a state holder that was not the chat transcript.

This was a Catch-22 that could not be resolved within the architecture. There was progress however, and sometimes the Agent was even capable of creating a whole semantic release. Not perfect, but it was clearly a step in the right direction. But as this architecture reached its limits, it, too, was shelved for another try.

3. The "bottom up tool chain" Agent entry portal

To address the confusion of too much tool call noise and choice overload, a new approach was to severely limit the Agent tool exposure to the absolute minimum. Here, the "bottom up" architecture was created. In this architecture, the agent had a tool list, but not one for him to call upon freely. It was the "target space" he needed to reach, prompted by a specific user intent.

If, let's say, the user wants to merge a database, the agent could then look up the tool "database_merge" within the tool list and call it. This tool then opened up a backwards pathway tool chain from which the agent could reach the desired endpoint "database_merge". The only thing the agent had to do then was follow the presented steps one by one while every completed step opened the "do next" step within the tool description list. No ambiguity here and no choice to be made by the agent. Well, almost no choices and almost no ambiguity.

Because the tool workflows aren't entirely linear end-to-end in an isolated chain, the backwards path was more of a web than a single line. This web had junctions where the Agent had to decide which directions to take. Let's say it landed in the taxonomy merge path and was given the choice to either keep the projections or merge them as well.

Depending on the user intent, the agent had to take a different route of validating the corresponding fingerprint of each projection.

But since the user intent was just the initial prompt, this intent got noisier and noisier down the path of the tool chain within the Agent context window. So the little distinction that the user wanted to merge the projections too (or that this required workflow decision was not expressed by the user initially) became an unstable instruction-following issue again, and the Agent still ended up in dead ends where it executed pipeline steps that did not lead to the end result and became unable to remedy the block on its own. For this, the Agent needed the capability to not only call tools, but also reset and modify the actual pipeline state to backtrack and choose a new way. At that point it became obvious that the missing thing was not another prompt rule but workflow state with resume and recovery semantics. This was in itself way too much context for the Agent to handle and would have resulted in a token-burning bonanza similar to the first approach.

So, this architecture was abandoned too. What followed was basically a synthesis of all the above.

4. The semantic control kernel

As it became clear that the Agent wasn't reliable enough to execute a complex tool chain, like, at all, the final approach was taken: do away with any Agent choices and only let him be the user-facing interaction surface and initial tool caller.

The MCP server now has 137 tool surfaces, but only 16 direct agent-facing tool calls to choose from. Everything else is represented as deterministic route and workflow definitions within the control kernel. This is not "hardcoded" in the cheap sense of hiding a mess, but in the governance sense: the possible paths are named, bounded, resumable and inspectable. This means the agent can choose one of 16 different actions, and the rest is deterministically controlled until the desired output is handed back to the agent so it only has to explain what was done. That's it.

But this approach brought another problem that had a bright and a dark side to it. In previous approaches, all the steps of creating taxonomies were LLM calls within the tool chain that were handled by the main Agent itself. But within the control kernel, there was no main agent that could process those steps. So, new LLM gateways were needed through which the control kernel could execute isolated LLM calls itself.

That meant an entirely new code branch within the kernel besides the deterministic step executions and an entirely new validation regime. Code-wise, a nightmare. Reliability-wise, a blessing, since isolated model calls were fine-tunable, free of any chat turn noise and always had a fresh and empty context window ready to be filled with much more sample data than the main agent could ever handle reliably. More importantly, those calls have a bounded job. They are not asked to "figure out the whole workflow". They are asked to produce one specific artifact or decision that the kernel can validate, store, reject or route forward.

And so the final architecture was developed: a fully deterministic workflow machine that incorporates and validates single LLM calls to structure and compose a full and rich semantic release, can merge, analyze, reset, rebuild databases and run the ingestion pipeline by itself, governed only by deterministic routes and junctions. The Kernel became the place where workflow truth lives: current state, next allowed step, user interaction request, progress, resume option, recovery option, receipt and final outcome. The Agent is reduced to a user-facing explainer that can extract user intent and make the initial tool calls to kick off the kernel workflows.

There is one very important boundary here: the Kernel does not become the owner of all domain truth. It owns the workflow reason for why something should happen and the state of that workflow, but the actual mutation still belongs to the module that owns the thing. If a Corpus Database needs to be created, reset, merged or rebuilt, the Kernel coordinates the route but the Corpus Builder owns the database primitive. If a Semantic Release needs to be compiled, validated or exported, the Normalizer owner contracts still own that domain. If a pipeline needs to run or runtime settings are needed, the Orchestrator remains the owner surface. The Kernel is the conductor, not the whole orchestra stuffed into one file. In hindsight, keeping the owner contracts as they were from the beginning made debugging the whole thing way easier because the failure modes were still locatable within the owners themselves.

As a side note: From the perspective of LLM-assisted development, this ownership structure became way more important than I expected. In the beginning it was just good modular hygiene. During the Kernel build phase it became a survival condition.

The constant loop of "fix bug A, but the fix creates bug B" only stayed manageable because failure modes were still contained inside relatively small owner modules. If everything had written into everything else, the LLM would have had to reason across an interconnected 200k line codebase every time something broke. That would have killed the project.

As it turned out, strict governance boundaries, hard file LOC limits and human-readable module structures were not old-fashioned bureaucracy, but the golden path for building a large system with AI assistance. The most referenced document in the whole process was my trusty `SPEC_Handover_Blueprint.md`, where the basic ownership rules, boundaries and working discipline were written down and enforced again and again, partly thanks to people constantly mocking "vibe coders messing things up" on social media platforms while providing me with valuable insight into the no-nos of software development and leading me to the construction of the handover blueprint that addresses all the pitfalls of messy, unreadable and un-debuggable code spaghetti.

One could say that this guardrail document made the whole build workable as a hard grindstone the LLM had to adhere to under all circumstances. Without it, The Machine would probably have fallen apart long before V1.

This also needs to be said because of how this thing was built. The Ontology Machine went from my personal "hello world" baseline to a working V1 in roughly six months of part-time,

LLM-assisted development. This is not mentioned as a victory lap, but because it explains why governance became such a central part of the architecture.

When a system grows that fast, the danger is not that too little code gets written. The danger is that too much code gets written without a stable memory of why anything exists and who owns what. The `SPEC_Handover_Blueprint.md` became the external spine of the project. It forced every build step back into owner boundaries, file-size limits, handover discipline and human-readable structure.

Without that, the speed that made the project possible would also have destroyed it. I am deeply grateful for all the mockery and objections from people regarding AI-assisted coding. Without them, I would have made many more mistakes and wasted a lot more time learning it the hard way and probably never reached V1.

Now let's get back to the owner boundaries. The same is true for the user interaction side. The Kernel owns the interaction contract, not the pixels. It can say "I need this target path", "I need this destructive confirmation", "this run is waiting for the user", or "this workflow can be resumed here". The Client Frontend renders that state, but the dialog truth stays in the Kernel. This matters because the Agent should not freely collect Kernel-required values in chat and then improvise a tool payload. The UI and Kernel own concrete values, confirmations and resume options. The Agent only explains them and selects the Kernel-facing workflow entrypoint.

This is also why the Kernel is not just a fancy batch script. A batch script runs until it falls on its face. The Kernel can deliberately block, ask for missing input, expose a resume option, classify a recoverable state, cancel a run or hand the user a support-visible failure. This fail-closed behavior is annoying when one only wants the thing to "just run" (and debugging that thing was a whole nightmare on its own), but it is the reason the workflow can survive restarts, partial state, stale locks, interrupted frontend sessions and user decisions that happen half way through a route.

The isolated LLM calls inside the Kernel have their own boundary too. The Kernel may render the prompt, define the expected artifact, validate the output and decide whether to retry, reject or route forward. But provider credentials and provider runtime selection are not Kernel truth. Those calls go through the Orchestrator-hosted runtime path, using the configured semantic control kernel LLM profile. This keeps model usage governed by the same credential and runtime system as the rest of the Machine instead of creating another hidden API key world inside the Kernel.

At the end of a Kernel workflow, the Kernel should know what happened well enough that the Agent does not have to invent a final story. Receipts, progress events and final explanation context exist for that reason. If something already existed before the run, the final answer should not pretend it was newly created. If a step was performed in this run, it can be explained as such. If the workflow blocked, the blocker should be named. This is one of those small-sounding things that becomes huge in practice because it stops the user-facing Agent from hallucinating the operational history of the Machine and exposes debug markers in a way that can actually be addressed.

At the end, one could ask why the agent is needed at all. If everything is Kernel-controlled, would a panel with 16 buttons be enough to control the whole creation/modify process? Well, yes, in principle. The point is to demonstrate how AI can be incorporated into a deterministic system as isolated working parts and user-facing interaction surfaces so "button pressing" isn't the core interaction pattern anymore and the functionality becomes accessible even to users that don't have prior knowledge of what the system can do.

The user-facing agent is not only a tool caller but can explain, guide and propose solutions to user questions about the system as a whole. A simple button panel can't do that.

Everyone that has worked with cryptic button labyrinths like Gimp, Blender, Unity or Unreal Engine knows exactly how it feels to learn the ropes of what button does what and how to chain those buttons together to get the desired result. The frustration is often not that one does not know what result one wants. The frustration is that one has to learn how the software is wired before one can even try to reach that result.

With the rise of AI-governed workflows, that interaction burden can be reduced dramatically. Not because expertise stops mattering, but because the user no longer needs to manually navigate every hidden procedural corridor of the software just to express intent. The control kernel is, in this sense, just a fancy unpolished demo of how this could be achieved.

9. The Client Frontend

Here is where the real value is created. Now, before we begin, let me say that everything that can be done within the client frontend is not exclusive to this surface in the sense that only the mighty client frontend is capable of doing this stuff. In fact, everyone could take the database and plug it into any tool-capable Agent Harness like OpenClaw, Codex, Claude Code, Hermes etc. and do the exact same stuff they can do here. The difference is that the interface within the Frontend is specifically tuned and the system prompts are specifically honed for the Agent to do exactly what it is supposed to do. It's a convenience thing for users that don't have or don't know how to work with their own agent harness. It's for everyone, not only for the Nerds that have a whole OpenClaw Agent swarm hooked up on a dozen cron jobs. In fact, if someone wants, they could plug their own Agent harness into the MCP server and play around with the internals of the whole Machine on their own and see what happens.

Now having said that, let's introduce the main chat surface.

It is designed in three sections and with a separate config suite for model credentials, path definitions and advanced tinkering. The main chat surface consists of a source list on the left side, the chat window in the middle and the page image viewer on the right. This is designed mainly for the query agent, since this is what its main purpose is supposed to be.

The Query Agent

The Frontend Query Agent has 15 read-only tools.

Tool	Purpose
<code>sql_query</code>	Run read-only SQLite <code>SELECT</code> / <code>WITH</code> queries against the active corpus DB.
<code>get_document_summary</code>	Load compact document identity, source-document context, promotions, structural hints and short excerpts.
<code>get_document_ontology_evidence</code>	Load compact ontology-facing evidence for lens and evidence-link work.
<code>get_document_rows</code>	Load row-focused material for tables, line items, orders, invoices and similar checks.
<code>get_document_provenance</code>	Load document-level provenance material when the exact target slot is not known yet.
<code>get_document_full</code>	Load the full document inspection bundle as an explicit last escalation step.
<code>get_document</code>	Legacy/full document read for compatibility and last-resort inspection.
<code>get_provenance</code>	Trace where a fact, field, slot, or promotion came from.

Tool	Purpose
<code>semantic_search</code>	Search by embedding similarity, with keyword fallback if embeddings are unavailable.
<code>database_coverage_snaps</code> <code>hot</code>	Get deterministic DB coverage, weak spots, promotion/field/row stats, and release state.
<code>list_source_documents</code>	List source-document groups created from page-level materialization.
<code>get_source_document</code>	Load a source document with its ordered page-level documents.
<code>list_ontology_lenses</code>	List available ontology lenses and their status/counts.
<code>get_ontology_lens</code>	Load an ontology lens with representative nodes, edges, and assertions.
<code>workbench</code>	Run restricted read-only local analysis via Python or PowerShell.

The Query Agent can inspect and analyze the active corpus, including base graph and ontology layers, but it cannot write to the database. Its system prompt can be inspected and modified within the config suite.

Most of those tools are structured read-only DB/repository access tools around the active corpus. They are not all free SQL tools. The only free SQL surface is `sql_query`, and even that is restricted to read-only `SELECT / WITH` statements. `semantic_search` uses embeddings when they exist and falls back to keyword search when they don't. The ontology and source-document tools are structured readers over the Base Graph and ontology layer. The `get_document_*` tools are a flat escalation ladder over the same repository reader: summary first, ontology evidence/rows/provenance next, full document only when the compact views are not enough.

If the normal read tools fail to deliver a result, the agent can use the workbench as a restricted read-only analysis surface. Python can be used for ad-hoc sqlite3 analysis through `MIN_AGENT_DB_PATH`; PowerShell is only allowed for narrow read-only corpus/config inspection under the active corpus scope. It is not a free code execution tool the agent can use to roam your machine, touch credentials, launch processes or meddle with the Windows runtime. It's basically a little escape hatch for analysis, not a god mode.

The query agent is designed with evidence in mind. It's not a simple "tell me what's in the Database" answer tool, but a workflow that delivers the evidence of its answers in a provable way that indicates how sure the agent is of its claims. This is done by a set of rules and checks, both within the prompt and in the code itself.

Within the agent's answer, if there are sources used to answer a query, the agent must give a clickable link to the source document in question. This link opens up the page image within the viewer on the right where the user can read the rendered ground truth and check it against the agent claim. Secondly, the same source is shown in the left source list. This list is compiled from tool results across tool call rounds, not just by reading the raw SQL text. The frontend collects source IDs from rows, results, fallback results, explicit tool result sources, `get_document_*` views and `get_provenance`. Naturally, this raw source set

can be way larger than what the agent then uses in its final answer. So, to not overwhelm the user with unrelated search tags, the source list is then compared to the source links the agent provides within its reply and only those sources are retained that are also present within the agent answer.

One might think this is a bit redundant, but it has a deeper purpose. When the Agent constructs a claim and lists a source that was not present in the current tool call turns, we end up with a mismatch between what the source list says and what the Agent says. It may happen that the agent hallucinates a source that it hasn't actually read for the query, or that it was a bleedover from previous chat turns that persists in the agent context window (which happens with weaker models way more often). In each case, we end up with more sources in the chat window than what the source list shows.

Now in order to not shift the burden onto the user to always count the source claims against the source list, every source from the agent that cannot be resolved against the source list is marked in red so the user can clearly see that the agent has produced an answer that either contains hallucinated facts or is referencing a source from within its context window that the agent hasn't actually read in the current turn (which is against its answer policy). Either way, the user knows that caution is advised.

As described in the corpus builder section, the design of the query agent and how it interacts with the database is as direct as possible. Only the read tools stand between the agent and the corpus. There is no hidden semantic answer validation here like in the Validator, except the source list match and the red unresolved-source marker. There are, however, hard technical safety borders: read-only SQL policy, row and text caps, workbench scope checks, output clipping and path restrictions. This might sound a bit too trusting, but through long and tedious testing it became clear that the LLM is surprisingly accurate when it can move freely through the evidence surfaces. Every semantic safety layer that is put in between the agent and the corpus decreases its expressiveness as well as its accuracy. This sounds paradoxical, but this is how it is. The more flexibly and freely the model can decide within read-only evidence access, the better the overall output became.

The Taxonomy Agent

This is the place where the user interacts with the control kernel and its workflows. The Taxonomy Agent exposes 16 Semantic Control Kernel tools.

Tool	Purpose
<code>empty_database_no_semantic_release</code>	Create a new Artifact Tree and empty Corpus DB shell without an active Semantic Release.
<code>empty_database_default_taxonomy_no_projections</code>	Create an empty DB with the default taxonomy, but without projections.
<code>empty_database_default_taxonomy_default_projections</code>	Create an empty DB with the complete default Semantic Release.

Tool	Purpose
<code>empty_database_default_taxonomy_custom_projections</code>	Create an empty DB with the default taxonomy and custom projections.
<code>empty_database_custom_taxonomy_no_projections</code>	Create an empty DB with a custom taxonomy staged, but without projections.
<code>empty_database_custom_taxonomy_custom_projections</code>	Create an empty DB with custom taxonomy and custom projections.
<code>manual_pipeline_run</code>	Run ingestion on source files for the active or selected database.
<code>database_merge_additive_only</code>	Merge two or more empty or filled databases additively into a new target database.
<code>database_rebuild_from_artifacts</code>	Rebuild a Corpus DB from an existing Artifact Tree and intact Semantic Release.
<code>create_custom_taxonomy_path</code>	Create a custom taxonomy from analyzed sample documents.
<code>create_custom_projection_path</code>	Create custom projections against an existing or staged taxonomy.
<code>reset_database</code>	Reset a database into an empty state through Kernel-approved policy.
<code>kernel_status</code>	Read current Kernel and Pipeline state without mutation.
<code>kernel_resume_state</code>	List workflows that can be resumed after interruption, blocking, or restart.
<code>kernel_continue_resumable_workflow</code>	Continue a resumable workflow using a Kernel-provided opaque resume option.
<code>kernel_cancel_active_run</code>	Request cancellation or pause of an active Kernel workflow or Pipeline run.

The job of the Taxonomy Agent is to delegate user intent into deterministic workflows and then explain what was done. It's as simple as that. The agent itself does not create the taxonomy by doing secret DB magic on its own. It picks the Kernel workflow that matches the user intent. The Kernel then owns the state, the dialog steps, blockers, resume options, receipts and the adapter calls into the owner modules.

`empty_database_custom_taxonomy_custom_projections`

`manual_pipeline_run`

`database_merge_additive_only`

`database_rebuild_from_artifacts`

`reset_database`

Those are the practical go-to calls in real usage. The support tools around them, `kernel_status`, `kernel_resume_state`, `kernel_continue_resumable_workflow` and `kernel_cancel_active_run`, are not creation paths but they matter a lot once a workflow runs for hours. They are how the agent can tell the user what the Kernel is doing, whether something can be resumed and how to stop an active run without poking random process state.

While there are many more creation pathways available, the fully custom path will be the workflow most users go for, simply because they will have their own data to work with and a fully custom release does not have the coverage issues of using the default Taxonomy. And why would a user go for just custom projections if they could just as easily create a full custom release? The effort is the same, and the outcome is way more precise than any other route they could take.

But the other creation pathways aren't there for fun either. For one, the fully custom route incorporates almost all of them into its own workflow at some point in its chain (it's the longest running chain), and therefore the other ones are needed anyway, so why not expose them too? Secondly, maybe someone wants a blank DB, just a taxonomy or a demo of the Master Default for other purposes, and this is an easy and quick way to get them.

If one combs the code, one might find remnants of a taxonomy modification path, but this path was abandoned due to the fact that it would have doubled the kernel size just for the benefit of modifying existing databases in-situ. And while this might be of use for those who have built up huge databases and want to change some things, it generally would not have justified the amount of work and code for covering some edge cases. So I kept just the creation route since most of the use cases can be accomplished with it.

To create a fully custom release, you just tell the Agent you want one or click the corresponding command within the dropdown menu and off it goes. The kernel will then ask you for a file path where the artifact tree should live, the name of the tree, the name of the database to be created and then for confirmation that the sample data has been put into the input folder of the artifact tree after it was created. Put the files into the folder, confirm and the process will run by itself. This happens before real ingestion. The sample files are used to author the Semantic Release, not as some hidden post-analysis of the final DB.

But there are two things to consider. First, the file path request from the kernel does not have a Windows folder picker, so one has to copy/paste the path into the request bar manually. This is the same for any workflow the kernel chews through. No convenient Windows folder picker, just copy/paste.

Secondly, the sample data you can provide is limited by the model's context window, so shoving a thousand-page book PDF into it might not be a good idea and will certainly end with a timeout error. There are no hard numbers on how much data you can put in as a sample, but from my experience, 50 pages still seems to be OK and will yield a decent result. One has to consider what the model does with the sample data.

It is used twice, once for creating the Taxonomy and a second time to create the projection. So, if one puts 50 pages of dense scientific text into it, the model has to churn through every page twice and this will take time and will create a massive token budget.

It is generally advised not to dump a single huge PDF into the input, but to split the sample data into single pages first and then pick the most relevant manually or, if it's like a book or a novel or something, aim for a random cross-sample size. The workflow generally works well for one-shot taxonomies in this way, but if the user wants to get more detail and more coverage, they can do more than one custom creation run on the same data to get multiple slightly different semantic releases with their own projection. This uses the fact that an LLM call never produces the same output and every call will focus slightly on different things and topics. The user can then merge all artifact trees together into one for a single semantic release and database to work with.

There are, however, tradeoffs to be considered while doing it this way. Every taxonomy merged will also introduce noise into the semantics because every call might use slightly different names for the same thing. The merge will de-duplicate doubles within the taxonomy and projections, but cannot reliably identify the same concepts named differently since it's a deterministic process. So you can end up with double meanings the normalizer later has to work with. Merging two releases is ok, three generally too, but with increasing numbers, the noise will grow and the output, and subsequently the database materialization, will carry that noise too.

`database_merge_additive_only`

The name says it all. The user can merge databases, but only additively, which means one can merge two or more empty databases with an active release without problems. One can also merge two or more filled databases together or a combination of filled and empty databases. Filled database merge preserves the materialized documents, page images, embeddings, evidence, promotions, entities, the Base Graph and also the ontology lenses that already exist in the source DBs. That last bit matters because ontology lenses are post-materialized knowledge and throwing them away during a merge would make the merge semantically stupid.

What you can't do is merge filled databases and also freely merge their projections into one new projection truth. That would destroy target IDs, Refs and fingerprints that are referenced during materialization and would leave tables without backlinks. With empty databases, merging projections works fine since there is no data to be referenced. So, if you want to merge full databases, projections must remain as they are, otherwise the merge will fail or should be blocked before it can do harm.

Also, merging filled databases will take time, depending on their size. Way more than one might expect, even on fast machines.

`database_rebuild_from_artifacts`

If the database gets corrupted or lost, the corpus builder can rebuild the database from the artifact tree content without any LLM call at any time. But this is not the same thing as a full DB snapshot restore. The Artifact Tree is the rebuild truth for the materialized corpus and the active Semantic Release. The deterministic Base Graph can be created again by running `basic_relation_mining` over the rebuilt DB. Ontology lenses are different. They are post-materialized DB knowledge created by the Ontology Agent. A plain artifact rebuild will not magically recreate them unless they were preserved somewhere else or exported/imported by a later workflow.

`reset_database`

This is not just a cute reset button even if it feels like one in the UI. It is a destructive Kernel-governed reset that brings the target corpus back to its empty state with an activated semantic release. It's useful if wrong data was ingested or if, for some reason, the user wants a fresh start without the need to create a new corpus through the creation path. The point is that the reset is done through policy and receipts, not by the Agent randomly deleting tables.

`manual_pipeline_run`

This starts the document ingestion process of The Machine. Put the files into the input folder and they will be written into the Database. You can stop and resume at any time since if a file is successfully ingested, it will be removed from the input folder and placed into the originals folder, so every file present within the input folder is, per workflow definition, not processed yet. If, for whatever reason, a file that is already ingested into the database gets copied into the input folder, the corpus builder will toss it out anyway because the hash is already present. This will cause no error, just a little notification.

The Ontology Agent

The Ontology Agent is the Knowledge Mining Layer of The Machine.

This needs a bit of explaining because the word "ontology" is wide enough to be almost useless if one just throws it into the room and expects the user to know what to do with it. Within The Machine, the Ontology Agent does not rebuild the corpus and it does not rewrite the materialized truth. The corpus DB already contains the evidence-bearing base: documents, page images, raw payloads, structured payloads, normalized data, evidence atoms, source documents, classifications, embeddings and the deterministic Base Graph.

The Ontology Agent works above that layer.

Its job is to create persistent, evidence-bound meaning layers over the same corpus. These layers are called ontology lenses. A lens is not just a note and not just a chat answer. It is a computable DB object made of terms, nodes, edges, assertions, evidence links, activation state and optional ontology embeddings. Basically fancy annotations on steroids.

This is the important part: A lens can represent a perspective without overwriting the base facts.

One lens can read a novel as story structure. Another lens can read the same novel as ruthless critique. Another lens can mark suspected extraction mistakes. Another lens can model a peer reviewer's interpretation of a paper. Another lens can classify a corpus from the perspective of a journalist, historian, scientist, lawyer, auditor or domain expert.

All of those lenses can live side by side over the same materialized corpus. They remain persistent, versioned, comparable and machine-readable. They do not vanish when the chat ends and they do not become dead scribbles in a comment field. They remain part of the DB and can be searched, compared, extended, exported and used by later Agents or human reviewers.

This is why the Ontology Agent is not simply "an ontology builder" in the narrow academic sense. It is a controlled way to turn user intent into semantic matter. The user brings the perspective, question, audit goal or review intent. The Agent translates it into a structured lens that can be checked against evidence later.

Why There Is No God Mode

It would be tempting to give the Agent full database write access and let it fix everything directly at the source. That would be easy, but it would break the main design promise of The Machine.

The base corpus is the materialized evidence trail. Even if it contains weak spots, review flags or extraction mistakes, those weaknesses belong to the history of how the corpus was created. If an Agent rewrites that layer freely, the system can no longer cleanly separate source materialization from later interpretation.

So the Ontology Agent does not overwrite the base facts.

If it finds something that looks wrong, it can create a correction, audit or review lens. That lens can say: "the materialized fact says X, but the evidence suggests Y." The contradiction remains visible and computable. The Query Agent can use such a lens in answers, but it must keep the base fact and the lens claim separated.

That is the safe version of God Mode: not rewriting the canon, but building a reviewable semantic layer above it.

Base Graph First

Before deeper knowledge mining makes sense, the database needs its deterministic Base Graph.

`basic_relation_mining` creates source-document groups from page-level materialization. It fills `source_documents`, `source_document_pages`, structural units and deterministic base

relations. It does not use an LLM and it does not guess. It uses source identity fields already materialized by the Corpus Builder.

The user can ask the Ontology Agent to run it. The tool runs against the active configured corpus DB; no separate database path is needed. Within the Client Frontend main window, below the LLM readiness, there is also a Base Graph status indicator. If it is red, the active database does not yet have a Base Graph. Tell the Agent to run the Base Graph and the Kernel will do the deterministic construction.

Tool Surface

The Ontology Agent exposes 16 tools.

It inherits 14 read-only Query Agent tools and adds 2 ontology/base-graph tools.

Tool	Access	Purpose
<code>sql_query</code>	Read	Run read-only SQLite <code>SELECT</code> / <code>WITH</code> queries against the active corpus DB.
<code>get_document_summary</code>	Read	Load compact document identity, source-document context, promotions, structural hints and short excerpts.
<code>get_document_ontology_evidence</code>	Read	Load compact ontology-facing evidence for lens and evidence-link work.
<code>get_document_rows</code>	Read	Load row-focused material for table-like documents and line-item checks.
<code>get_document_provenance</code>	Read	Load document-level provenance material when the exact target slot is not known yet.
<code>get_document_full</code>	Read	Load the full document inspection bundle as a last escalation step.
<code>get_document</code>	Read	Legacy/full document read for compatibility and last-resort inspection.
<code>get_provenance</code>	Read	Trace where a fact, field, slot, or promotion came from.
<code>semantic_search</code>	Read	Search by embedding similarity, with keyword fallback if embeddings are unavailable.
<code>database_coverage_snapshot</code>	Read	Get deterministic DB coverage, weak spots, promotion/field/row stats, and release state.
<code>list_source_documents</code>	Read	List source-document groups created from page-level materialization.
<code>get_source_document</code>	Read	Load a source document with its ordered page-level documents.
<code>list_ontology_lenses</code>	Read	List available ontology lenses and their status/counts.

Tool	Access	Purpose
<code>get_ontology_lens</code>	Read	Load an ontology lens with representative nodes, edges, and assertions.
<code>basic_relation_mining</code>	Kernel write	Run deterministic Base Graph/source-document/structural-unit construction on the active configured corpus DB.
<code>sql_batch_execute</code>	Ontology write	Execute an atomic, preflight-validated batch of ontology-layer SQLite writes.

Write Scope

`basic_relation_mining` writes deterministic Base Graph structures only.

`sql_batch_execute` may write ontology-layer objects such as:

- ontology lenses
- ontology terms
- ontology nodes
- ontology edges
- ontology assertions
- ontology evidence links
- ontology activation state
- ontology run/checkpoint state
- ontology-scoped source-document classifications

It must not overwrite deterministic `base` or `semantic_release` classification rows.

What makes this powerful is not that an AI writes something into a DB. The powerful part is that the written meaning remains evidence-bound, versioned and computable. Let's say you love novels and want to write yourself, but you don't know how good stories are constructed. Throw your favorite novel into the database and ask the Ontology Agent to create a lens from the perspective of a master novel writer. The Agent can take apart the novel, define its story arc, characters, relationships, emotional tones, conflicts, archetypes, weak points, pacing and whatever else belongs to that lens.

Then you can create another lens from the perspective of a reader. The Agent can model where tension builds, where expectation is created, where filler starts to feel like filler, and what kind of reading experience the structure creates.

Or you can create a ruthless critique lens and let the Agent tear apart the same text from that angle.

The same principle applies outside literature. A journalist can mine ideological patterns across a news corpus. A scientist can let multiple reviewers build separate review lenses over the same paper corpus. An investigator can build event, actor and contradiction

lenses. A company can build audit lenses over invoices and delivery notes. A private person can build a tax lens over paperwork.

In the peer review case this becomes especially interesting. Each reviewer can get their own lens. The lenses can then be compared against each other to see where reviewers agree, where they contradict each other and which patterns survive across independent interpretations. This is not a bunch of scattered comments anymore. It is a computable review artifact that stays useful for humans and AI systems.

At this stage of development, with The Machine almost finished, the sample DBs created and the last test runs going, something not so nice turned up. During the whole design process, token efficiency was closely observed and optimized at every corner. During the creation of the Ontology Agent I naturally assumed that its token usage would be slightly higher than that of the Query Agent because it writes into the DB. What I did not anticipate was that, for the Agent to gather enough context to actually make a useful `batch_execute`, it needs way more context than the Query Agent. And with "way more" I mean it.

During a stress test of the Ontology Agent on the sample DB, I noticed a 4 minute agent turn and subsequently put a little token counter into the Frontend UI to see what was going on. As it turned out, there were so many tool rounds and model calls going on that one Agent turn consumed a staggering 500k tokens.

This was obviously unacceptable. At this pace, a full Ontology over a relatively small database would easily reach 10, 20, 30 million tokens. Something that would make every bank account weep.

So, something had to be done. The search tools were extended with the new `get_document_*` escalation ladder so the Agent could query without getting a full document dump, context baggage was reduced and compacted during failed/successful writes and indeed, the token usage went down from those 500k peaks to 150k token peaks without noticeably losing agent capability. But here is where the optimization ends without cutting into vital context of the Agent work.

Here we must make a clear distinction. Ontology work is not for corpora of more than a couple of hundred pages. 100 pages seem to be the sweet spot for broad ontology work: expensive enough to be noticed, but not yet the kind of thing that burns a hole into your pocket. At 300 to 500 pages, we are rapidly approaching prohibitively expensive territory if one wants to create an ontology that covers the whole corpus. A thousand pages are basically not feasible, not only because of the token usage but also because of the batch write capabilities of the Agent, which can reliably write into 5 or at most 10 pages per turn without running its head against the validation gate.

The ingestion pipeline easily chews up thousands of pages and the Query Agent can work with large databases. The Base Graph can be created, but for serious Ontology work, version 1.0 of The Machine is currently restricted to small specialized corpora like papers, small article collections, books and such.

To make large-scale mining feasible, token prices must come down substantially and the Ontology Agent needs a broad and flexible "Workbench" like mode in which the database

can be handled and edited in arbitrary large chunks. This is currently not the case and lies outside the scope of what I am willing to develop, simply because it would be another massive project on its own.

And why this is so will become clear if one considers the following safety model for the Ontology Agent. The small write chunks are a direct result of current model capability restrictions. Even with 5 documents in a batch write, the Agent regularly needs 2 or more tries to pass all checks before the write can enter the database. Now consider how the Agent would spin its wheels if it needed to get 100 page edits passed. For that to work, we probably need a couple more model generations.

Safety Model

`sql_batch_execute` enforces:

- ontology-layer write allowlist
- explicit primary IDs
- parent-first writes
- required JSON defaults
- evidence target checks
- same-lens reference checks
- node-to-node edge endpoint checks
- deterministic preflight validation
- ontology edit logging
- post-write validation
- ontology embedding refresh when possible

The Ontology Agent can modify ontology meaning, but it does not have free write access to the full corpus DB.

The safety model is deliberately boring and strict. Before a write enters the database, the tool checks whether the Agent is touching only the ontology/relation layer it is allowed to touch, whether all required IDs exist, whether parent objects exist before child objects, whether edge endpoints point to nodes, whether evidence links point to valid targets and whether JSON fields contain valid defaults instead of `NULL` garbage.

The preflight runs before the transaction opens. If it finds a repairable mistake, the Agent treats this as internal tool feedback, inspects schema or IDs if needed and tries a corrected `sql_batch_execute` within the repair budget. The user should not get every failed draft dumped into the chat as if it were a final answer. Only if the repair budget is exhausted, or if the failure is not repairable, should the Agent tell the user what blocked the write.

This is not a normal code bug. This is the nature of letting a non-deterministic model create deterministic DB objects. The system does not trust the model blindly. It makes the model pass the gate.

The chunk size is therefore not only a performance limitation. It is also part of the safety model. The Agent cannot write a full ontology in one shot, and it should not pretend that it can. It builds the lens bit by bit, with validation after each edit unit, while the user keeps control over the intent and scope.

It must be stated clearly, this is an interactive user/AI co-creation process, not a "one-shot AI does something magic".

Because in the 21st century there is no magic, there is only code.

10. Use Case Examples

This section is not meant as a cute little "you can search your documents" list.

That would be underselling the whole thing.

The Machine is interesting because it can turn arbitrary document piles into a custom semantic database, keep the evidence trail intact, then let humans and AI build persistent knowledge layers above it. That means the real use cases are not just "ask questions about PDFs". The real use cases are: create a domain-specific truth surface, query it, mine it, compare perspectives over it and keep the whole process evidence-bound.

The examples below are not science fiction. Some are expensive, some are prohibitively expensive, some are politically spicy, some need good data hygiene, a lot of compute, and a good chunk of them are so time consuming and so expensive that currently only institutions will have the resources to execute them.

but all of them follow from the architecture that already exists

Media Worldview Analysis

A journalist could ingest months or years of news transcripts, broadcast subtitles, articles, press releases and fact sheets.

The custom taxonomy would not have to be "news" in a generic sense. It could model:

- topic framing
- actor roles
- threat narratives
- victim/aggressor language
- moral vocabulary
- omitted context
- geopolitical alignment
- emotional temperature
- source attribution
- recurring argument patterns

The Query Agent can then answer normal questions like "which topics dominated March?" or "how often was actor X described as dangerous?".

The Ontology Agent can go further and build lenses like:

- government trust lens
- ideology/worldview lens

- war framing lens
- economic anxiety lens
- moral panic lens
- soft propaganda lens
- cross-country comparison lens

Now one can compare a week of one broadcaster against six months of another, or compare German, American and British coverage of the same events. The point is not that the Agent says "this is propaganda". The point is that every claim can be backed by source documents, page images, evidence atoms and a persistent lens that can be inspected or challenged.

This is where a small independent analyst suddenly gets tooling that used to require a research institute.

Peer Review As Computable Knowledge

Scientific peer review is usually a mess of PDFs, comments, emails and authority.

With The Machine, every reviewer could get their own ontology lens over the same paper corpus. One reviewer builds a methods lens. Another builds a statistics lens. Another builds a novelty lens. Another builds a replication-risk lens. Another builds a literature-gap lens.

The result is not a bunch of dead scribbles.

It is a set of evidence-bound, comparable, computable review artifacts.

One can then mine across the reviewer lenses:

- where do reviewers agree?
- where do they contradict each other?
- which criticism is evidence-bound?
- which criticism is taste or authority?
- which claims survive across independent lenses?
- which papers repeatedly trigger the same methodological weakness?

This would not replace human review. It would make the review process less priestly and more inspectable. A reviewer would still think, judge and argue, but the result of that work would become a structured artifact that later humans and AI systems could build on.

That is a pretty big deal.

Legal Discovery And Case Memory

Legal corpora are basically built for this.

Contracts, emails, invoices, witness statements, court filings, exhibits, internal memos, policies, settlement drafts, meeting minutes and scanned documents can all become one searchable corpus.

The custom taxonomy can model:

- parties
- obligations
- deadlines
- claims
- counterclaims
- contractual clauses
- risk language
- exceptions
- contradictions
- document provenance

The Base Graph matters here because long documents must remain reconstructable as source documents, not just as loose pages.

Ontology lenses can then model:

- plaintiff theory
- defense theory
- contradiction map
- witness reliability map
- timeline of events
- obligation breach map
- settlement leverage map

The same evidence can be viewed from different legal strategies without rewriting the underlying corpus. That means one can build a case theory lens, then build the opposing case theory lens, and compare both against the same evidence surface.

Corporate Audit, Procurement And Fraud Detection

The invoice/order/shipping demo already points into this direction.

Take purchase orders, invoices, delivery notes, emails, receipts, approvals, payment exports and vendor master data. Build a release that understands the local business semantics instead of forcing everything into some generic accounting schema.

The Machine can then expose:

- invoice/order/shipping mismatches
- missing delivery evidence
- unusual approval chains
- recurring vendor anomalies
- duplicate-looking transactions
- price drift
- repeated manual overrides
- late delivery patterns
- entities that appear across unrelated document groups

The Ontology Agent can create audit lenses:

- high-risk vendor lens
- missing evidence lens
- suspicious timing lens
- procurement leakage lens
- duplicate payment suspicion lens
- policy violation lens

Important: those are not changes to the accounting truth. They are reviewable semantic claims above the materialized facts.

That makes it possible to say: "the source says this invoice is valid, but the audit lens says the surrounding evidence is weak."

Intelligence And OSINT Dossiers

For open source intelligence, the interesting thing is not one document. It is the web of documents.

Articles, archived pages, leaked PDFs, company registers, sanctions lists, court files, public procurement records, social media exports, speeches and biographies can be materialized into one corpus.

The Machine could then build:

- actor networks
- organization maps
- event timelines
- ownership structures
- contradiction lenses
- rumor vs verified evidence lenses
- source reliability lenses
- geopolitical influence lenses

The system does not need to decide one final truth. It can preserve multiple competing lenses:

- "what the public record proves"
- "what the hostile source claims"
- "what independent sources confirm"
- "what remains speculative"

That separation is exactly why the ontology layer matters. It keeps interpretation visible instead of smearing it into the base data.

Investigative Journalism Over Large Archives

Imagine a journalist gets a leaked archive with 100,000 files.

Normally the first problem is not "what is the answer?" but "what the hell is in here?".

The Machine can create a first custom release from samples, ingest the archive, build the Base Graph, then let the Query Agent surface coverage, weak spots and suspicious clusters. After that, the Ontology Agent can build investigation lenses:

- actor/event map
- money flow lens
- internal contradiction lens
- document authenticity concern lens
- timeline lens
- source confidence lens
- unresolved questions lens

The good part is that the journalist does not have to know the schema beforehand. The custom taxonomy is created from the material itself. That is the difference between forcing

a leak into a prebuilt database and letting the archive tell you what semantic shape it wants to have.

Literature, Storycraft And Editorial Surgery

Throw novels, scripts, short stories, screenplays or game writing into the Machine.

A writing taxonomy can model:

- characters
- arcs
- motivations
- scenes
- pacing
- conflict
- foreshadowing
- emotional turns
- exposition density
- point of view
- dialogue function
- theme

Then build lenses:

- master novelist lens
- ruthless editor lens
- reader experience lens
- genre convention lens
- pacing failure lens
- character agency lens
- adaptation lens

A writer could compare their own manuscript against books they love, not by asking "is it good?" but by mining how the structure behaves. Where does tension rise? Where does the story stall? Which characters vanish? Which promises are never paid off? Which scenes do no work?

The lens does not have to flatter the author. It can be built to be brutal.

That is useful.

Theology, Scripture And Tradition Mapping

The Bible example is obvious, but the idea extends to any large canonical text.

If the corpus has a good base structure, like canon -> testament -> book -> chapter -> verse -> pericope, the Machine can support serious comparative work.

Possible lenses:

- narrative arc lens
- theological theme lens
- legal command lens
- prophecy fulfillment lens
- character/lineage lens
- translation divergence lens
- doctrinal tradition lens
- church father reference lens
- contradiction/interpretation lens

Different traditions can have their own lenses over the same base text. Protestant, Catholic, Orthodox, Jewish, academic-historical, literary and skeptical lenses can all exist side by side.

That is not "the AI explains the Bible".

It is a structured space where interpretations become evidence-bound objects that can be compared.

Medical Record Review And Care History

This one must be handled carefully because medical use is high risk, but as a document analysis system the pattern is clear.

A patient record can include lab reports, imaging reports, discharge letters, medication plans, doctor notes, referrals, insurance letters and patient diaries.

The Machine could build:

- timeline of symptoms
- medication change map
- diagnosis history
- unresolved findings lens
- contradiction between reports
- missing follow-up lens

- patient narrative lens
- second-opinion preparation lens

It should not replace medical judgement. But it could help a patient or physician see what is actually documented, what is missing and where the record contradicts itself.

The evidence-bound design matters here because a medical claim without source trace is worthless or dangerous.

Personal Life Archive And Memory Machine

A private person could ingest letters, scanned notebooks, emails, photos with OCR, contracts, school records, creative drafts, medical files, old journals, family documents and financial paperwork.

This becomes more than search.

The Machine could build lenses like:

- family history lens
- legal/contract obligations lens
- financial paperwork lens
- biography timeline lens
- recurring life themes lens
- unresolved administrative tasks lens
- "what should my heirs know?" lens

This sounds small compared to corporate use cases, but it may be one of the most human ones. Most people do not have a knowledge-management problem because they lack data. They have it because their life is spread across folders, PDFs, scans and forgotten documents.

University And Research Stack

A researcher can ingest papers, notes, dataset descriptions, code docs, reviews, annotations, draft chapters and correspondence.

The custom release can be tailored to the research field. A historian does not need the same taxonomy as a chemist, and neither needs the same taxonomy as a media scholar.

Ontology lenses can model:

- literature map
- method map

- theory lineage
- disagreement graph
- evidence strength
- open questions
- replication concerns
- citation clusters
- "paper X through theory Y" lens

A PhD student could build a living literature review that is not just a Zotero folder and a pile of highlighted PDFs, but a computable semantic corpus that keeps the source evidence attached.

Education And Curriculum Mining

Schools, universities and training companies can feed curricula, textbooks, exams, lecture notes, rubrics and student submissions into The Machine.

The Machine can then expose:

- skill coverage
- topic gaps
- repeated misconceptions
- assessment alignment
- curriculum drift
- grade-level mismatch
- hidden prerequisites
- redundancy across courses

Ontology lenses can represent:

- teacher view
- student confusion view
- exam preparation view
- accreditation view
- mastery progression view

This is much more interesting than "generate a lesson plan". It lets an institution understand its own educational material as a living semantic system.

Product Support And Field Failure Memory

A company can ingest support tickets, manuals, logs, forum posts, warranty claims, internal repair notes and customer emails.

Possible lenses:

- product failure map
- recurring complaint lens
- support answer quality lens
- known workaround lens
- documentation gap lens
- feature confusion lens
- "what users actually try to do" lens

The important bit is that the Machine can preserve the messy evidence rather than flatten it into a clean dashboard too early. Support reality is full of contradictions, weird edge cases and partial explanations. Ontology lenses can hold those patterns without pretending they are final truth.

Regulatory And Compliance Memory

Regulations, internal policies, audit reports, compliance evidence, incident reports, vendor documents and procedure manuals can be ingested into one corpus.

The custom taxonomy can model obligations, controls, exceptions, responsible parties, deadlines, evidence requirements and risk categories.

The Ontology Agent can then build:

- obligation map
- evidence coverage lens
- missing control evidence lens
- policy conflict lens
- audit readiness lens
- regulator question lens

This helps because compliance is usually not a lack of documents. It is a lack of traceable relation between what must be true, what is claimed to be true and what evidence actually exists.

Software Architecture And Codebase Archaeology

The Machine is not limited to PDFs.

It can ingest handover docs, specs, tickets, logs, architecture notes, READMEs, generated reports and code documentation. With the right release, it can create a semantic memory of a software system.

Possible lenses:

- ownership boundary lens
- risk area lens
- flaky subsystem lens
- undocumented behavior lens
- architecture drift lens
- release readiness lens
- "what the docs claim vs what the code does" lens

This is especially relevant for AI-assisted coding. If LLMs are going to work on large systems, they need bounded truth surfaces. A corpus that can preserve code-adjacent evidence and build computable interpretation layers is exactly the kind of structure that keeps an AI from drowning in a giant repo.

Market And Competitor Intelligence

Companies can ingest competitor websites, pricing pages, product docs, reviews, support forums, changelogs, job ads, investor reports and press releases.

The Machine can build:

- product positioning map
- pricing movement lens
- strategic priority lens
- customer pain lens
- feature gap lens
- hiring signal lens
- market narrative lens

This is not about scraping one page and asking for a summary. It is about building an evidence-backed semantic memory of a market and watching it evolve.

Cultural Pattern Mining

Books, movies, reviews, essays, social media exports, speeches, games, comics and fan discussions can be treated as a cultural corpus.

Ontology lenses can model:

- archetypes
- social anxieties
- political metaphors
- gender roles
- class signals
- religious motifs
- technological fear
- hope and despair patterns
- generational language

This can be used seriously in humanities research, but also creatively by writers, designers and worldbuilders. A worldbuilder could mine myths, histories and novels, then build a lens that extracts social structures, taboos, rituals and power relations for inspiration.

Memory Core For Evolving Agents

This is probably one of the stranger, but also one of the coolest implications.

If the corpus DB can hold documents, evidence, memories, events, relations and ontology lenses, then it can also become a Memory Core for an Agent.

Not just a vector memory bucket.

A real structured memory surface.

An Agent could write or receive memory events into the corpus. Conversations, decisions, mistakes, promises, preferences, emotional traces, important user facts, world events, NPC experiences or game state history could all become materialized memory objects. The Base Graph gives order and source relation. The ontology lenses give interpretation.

The interesting part is that personality is then not just a prompt.

It becomes a lens over memory.

The same memory corpus can be retrieved through different active lenses:

- cautious lens
- impulsive lens
- brave lens

- bitter lens
- hysterical lens
- neutral analyst lens
- loyal companion lens
- paranoid survival lens
- wounded character lens
- curious explorer lens

The memories do not have to change. The interpretation and retrieval frame changes.

A cautious Agent might retrieve failures, risks and broken promises more strongly. A brave Agent might retrieve moments of success, trust and bold action. A bitter NPC might interpret the same past event as betrayal. A loyal companion might interpret it as a misunderstanding. A neutral assistant might suppress emotional weighting and surface only the factual chain.

That means an evolving assistant, game NPC or companion character could have persistent memory, but also a controllable personality layer over that memory. The Agent does not need to pretend it has a personality only because the system prompt says so. It can use a structured lens to decide what memories matter, how they relate to each other and how they should color the current response.

For NPC design this is wild.

You could create a village full of characters that share the same world memory, but every character has their own lens. The coward remembers danger. The mayor remembers obligations. The old soldier remembers patterns of war. The child remembers wonder. The traitor remembers opportunity. The priest remembers guilt and meaning.

Same world.

Different memory lens.

Different personality.

For companion design it is equally interesting. A companion could evolve over months because the memory corpus grows. Instead of stuffing everything into a prompt or a flat vector store, the Agent can mine its own remembered life through active lenses and persistent ontology structures. It can have review lenses, correction lenses, trust lenses, emotional-history lenses and preference lenses.

This is compute-intensive and not something one casually runs on a potato laptop with a million memories. But conceptually it is a very strong use case because it turns memory from "retrieve similar snippets" into "retrieve structured experience through a chosen interpretive frame".

That is a Memory Core.

Why These Use Cases Fit The Machine

All those examples have the same skeleton:

1. There is a messy document pile.
2. The document pile does not fit a generic schema.
3. The user needs a domain-specific semantic release.
4. The result must remain evidence-bound.
5. The user needs more than one perspective over the same facts.
6. Those perspectives must persist beyond a chat answer.
7. Humans and AI should both be able to build on the result.

That is the niche.

The Machine is not a chatbot with file upload. It is not a vector search wrapper. It is not a dashboard generator. It is a way to produce a computable semantic corpus and then mine knowledge layers over it.

That is why the architecture is weirdly large for something that, on the surface, could look like "ask questions about documents".

Because that is not what it is.